

Lab Sheet 03

Title: Interfacing Sensors with ESP32 in Wokwi

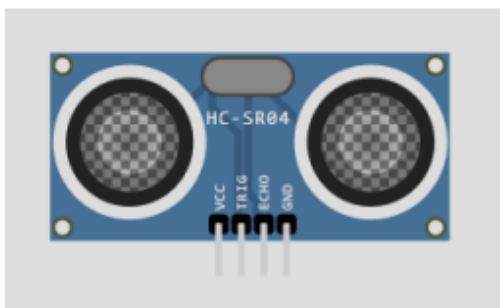
Aims: Learn how to correctly connect various sensors to the ESP32 Devkit in the Wokwi online simulator. This lab sheet focuses only on hardware connections (wiring) and includes a brief explanation of each sensor's purpose.

Tasks:

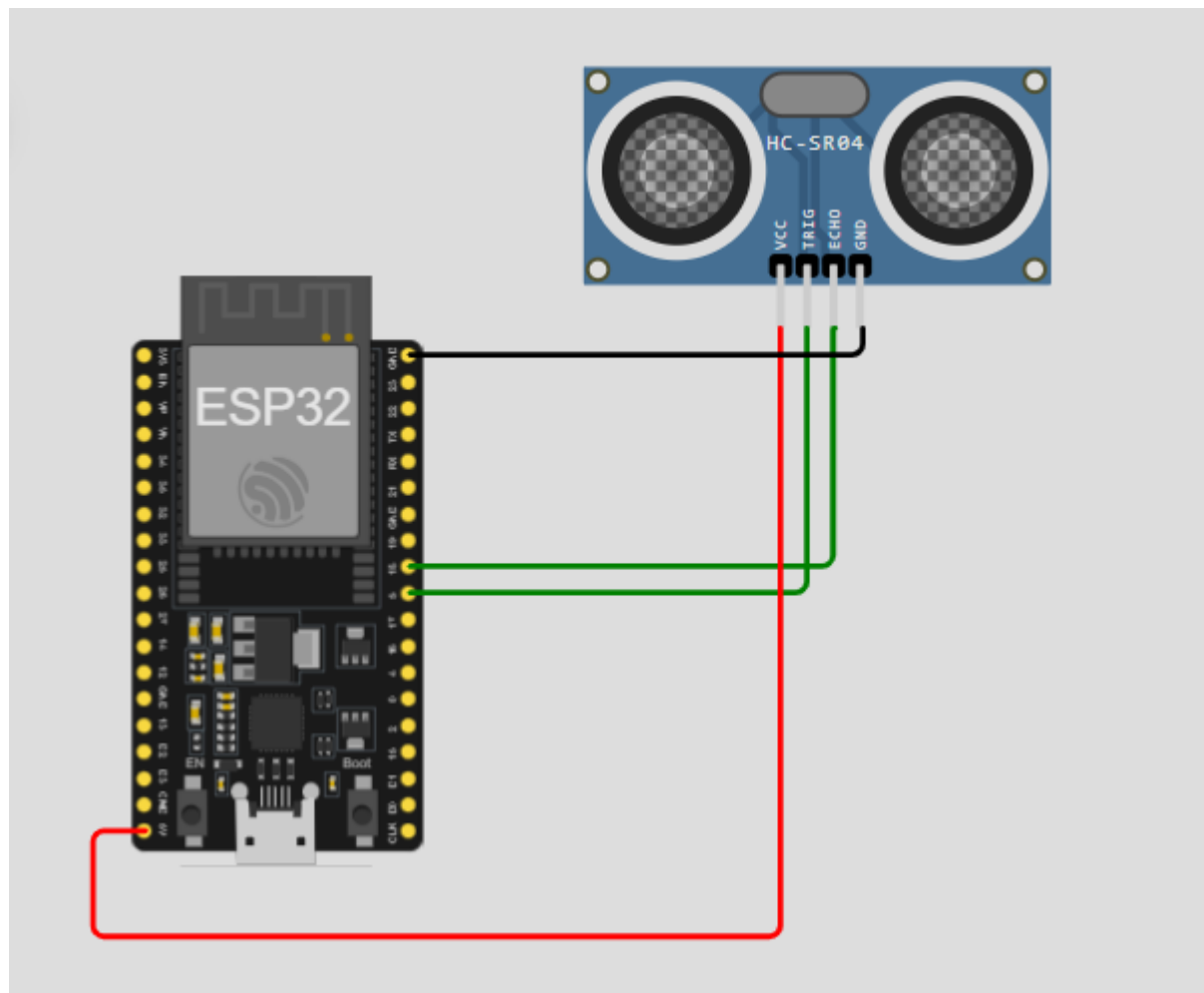
- Open wokwi.com and start a new project with ESP32 DevKit.
- Use the + button (Add Part) to search and add each sensor.
- Connect using wires (click and drag from pin to pin).
- Power: Most sensors use 3.3V or 5V (check individual notes). ESP32 GPIO pins are 3.3V logic.
- Always connect GND properly. Use recommended pins where possible for easy compatibility.

01.HC-SR04 (Ultrasonic Distance Sensor)

This sensor measures the distance to an object without physical contact by emitting ultrasonic sound waves and calculating the time it takes for the echo to return. It is commonly used in robotics for obstacle avoidance, parking assistance, level measurement, and interactive projects.



Pins on Sensor	Pins on ESP32
VCC	5V
TRIG	GPIO 5
ECHO	GPIO 18
GND	GND



```
#include <Arduino.h>

// Define pins
const int trigPin = 5;
const int echoPin = 18;

// Define sound speed in cm/uS
#define SOUND_SPEED 0.034

Long duration;
float distanceCm;

void setup() {
  Serial.begin(115200); // Start serial communication
  pinMode(trigPin, OUTPUT); // Sets the trigPin as an Output
  pinMode(echoPin, INPUT); // Sets the echoPin as an Input
}
```

```

void loop() {
  // Clears the trigPin
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);

  // Sets the trigPin on HIGH state for 10 micro seconds
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

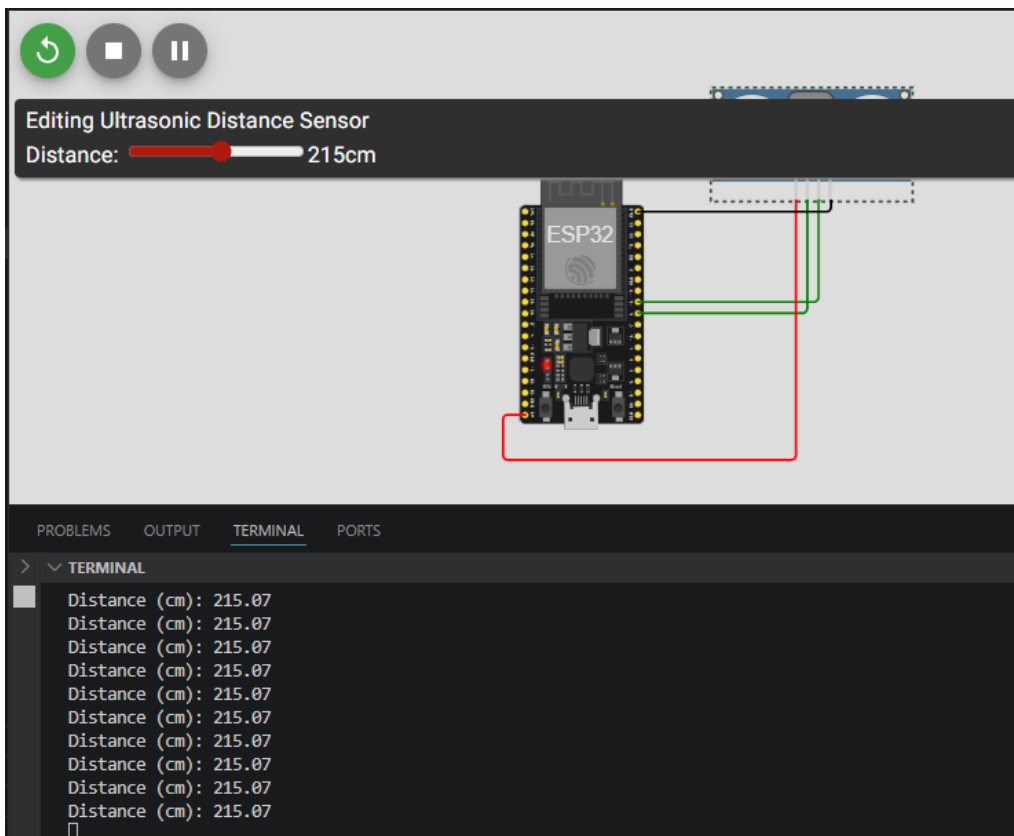
  // Reads the echoPin, returns the sound wave travel time in microseconds
  duration = pulseIn(echoPin, HIGH);

  // Calculate the distance
  // Distance = (Time x Speed of Sound) / 2
  distanceCm = duration * SOUND_SPEED / 2;

  // Prints the distance on the Serial Monitor
  Serial.print("Distance (cm): ");
  Serial.println(distanceCm);

  delay(500);
}

```

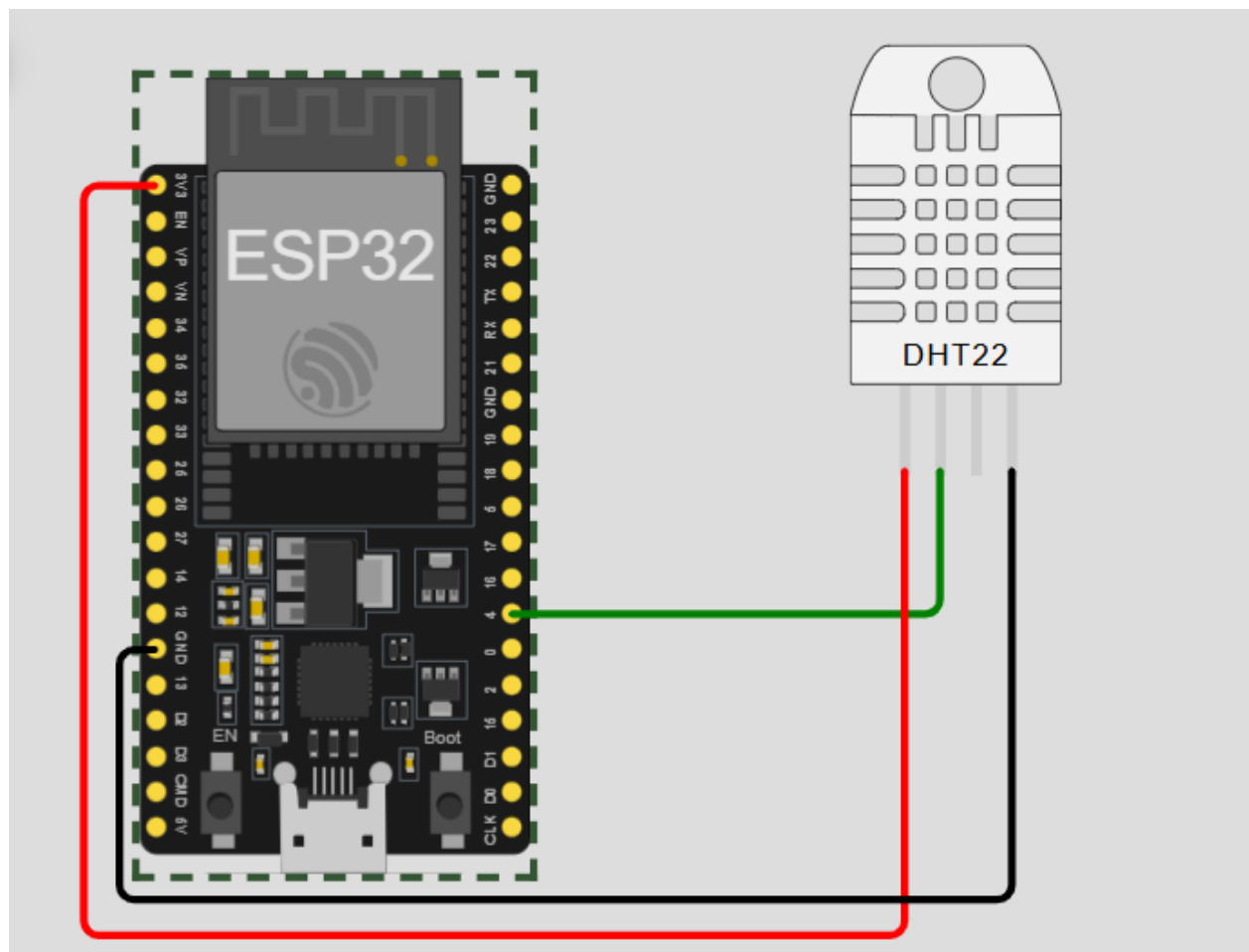


02.DHT22 (Digital Humidity & Temperature Sensor)

It measures ambient temperature and relative humidity with good accuracy. Widely used in weather stations, environmental monitoring, HVAC systems, and smart home automation.



Pins on Sensor	Pins on ESP32
VCC	3.3V
SDA	GPIO 4
NC	-
GND	GND



```
#include <Arduino.h>
#include "DHT.h"
```

```
#define DHTPIN 4    // Digital pin connected to the DHT sensor
```

```

#define DHTTYPE DHT22    // DHT 22  (AM2302), AM2321

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(115200);
  Serial.println(F("DHT22 test!"));

  dht.begin();
}

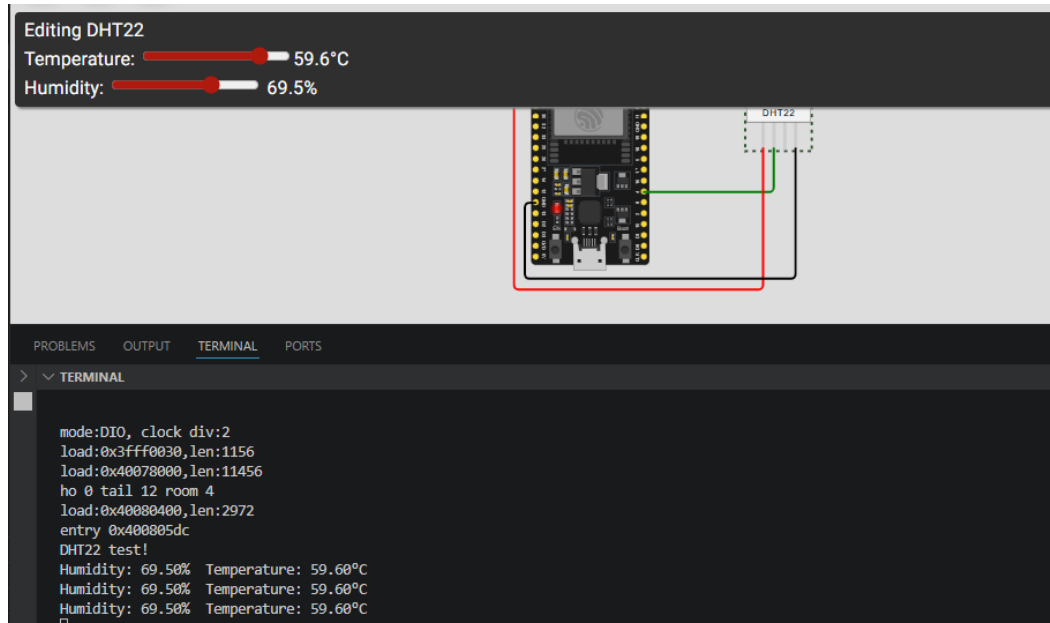
void loop() {
  // Wait a few seconds between measurements
  delay(2000);

  // Reading temperature or humidity takes about 250 milliseconds!
  float h = dht.readHumidity();
  // Read temperature as Celsius (the default)
  float t = dht.readTemperature();

  // Check if any reads failed and exit early (to try again).
  if (isnan(h) || isnan(t)) {
    Serial.println(F("Failed to read from DHT sensor!"));
    return;
  }

  Serial.print(F("Humidity: "));
  Serial.print(h);
  Serial.print(F("%  Temperature: "));
  Serial.print(t);
  Serial.println(F("°C "));
}

```

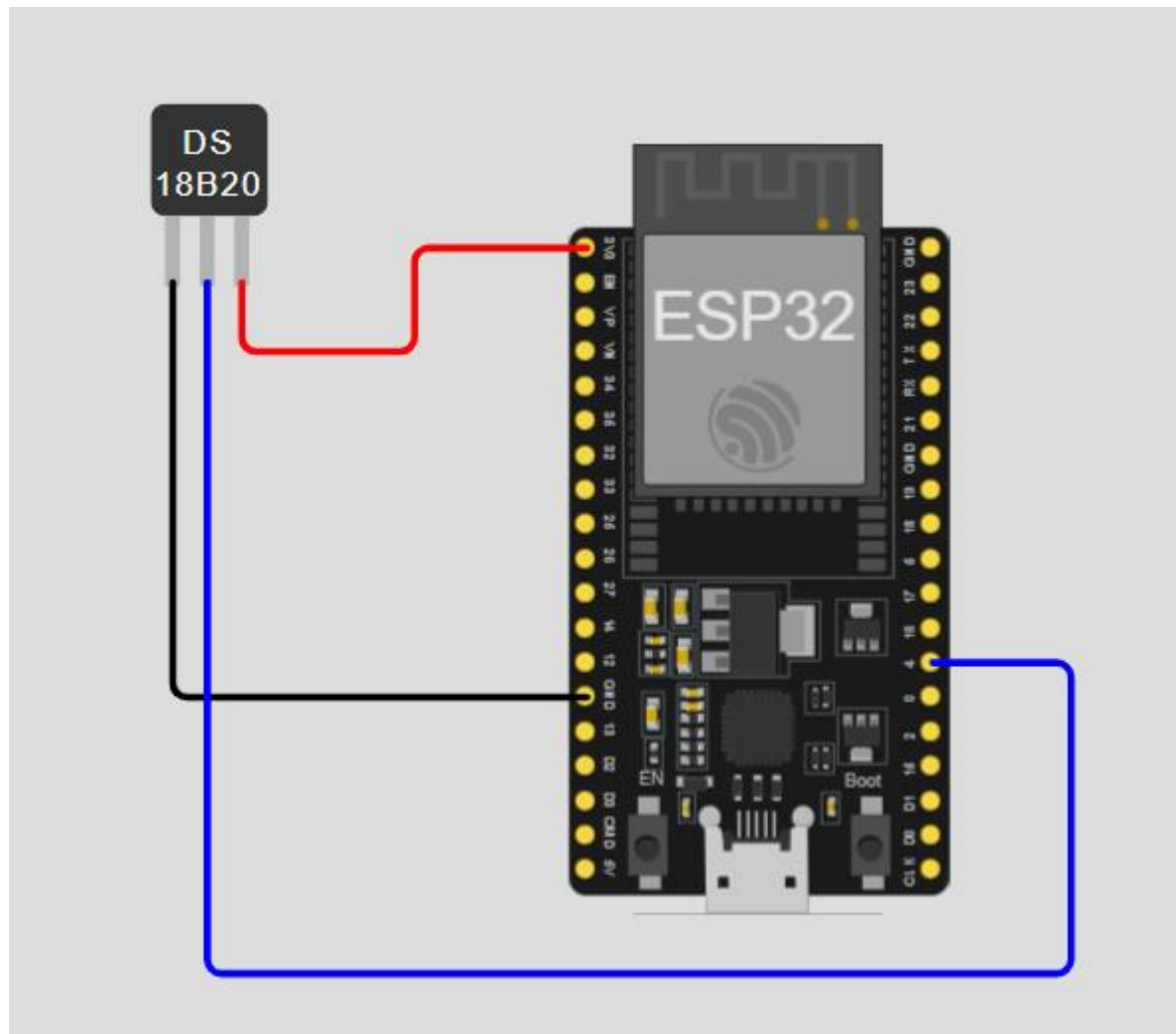


03.DS18B20 (One-Wire Digital Temperature Sensor)

Provides precise digital temperature readings. Ideal for applications requiring accurate temperature monitoring (e.g., incubators, aquariums, industrial process control, or multiple-point temperature sensing) because multiple sensors can share one wire.



Pins on Sensor	Pins on ESP32
GND	GND
DQ	GPIO 4
VCC	3.3V



```
#include <Arduino.h>
#include <OneWire.h>
```

```

#include <DallasTemperature.h>

// Data wire is plugged into port 4 on the ESP32
#define ONE_WIRE_BUS 4

// Setup a oneWire instance to communicate with any OneWire devices
OneWire oneWire(ONE_WIRE_BUS);

// Pass our oneWire reference to Dallas Temperature.
DallasTemperature sensors(&oneWire);

void setup() {
  Serial.begin(115200);
  Serial.println("DS18B20 Temperature Sensor Test");

  sensors.begin();
}

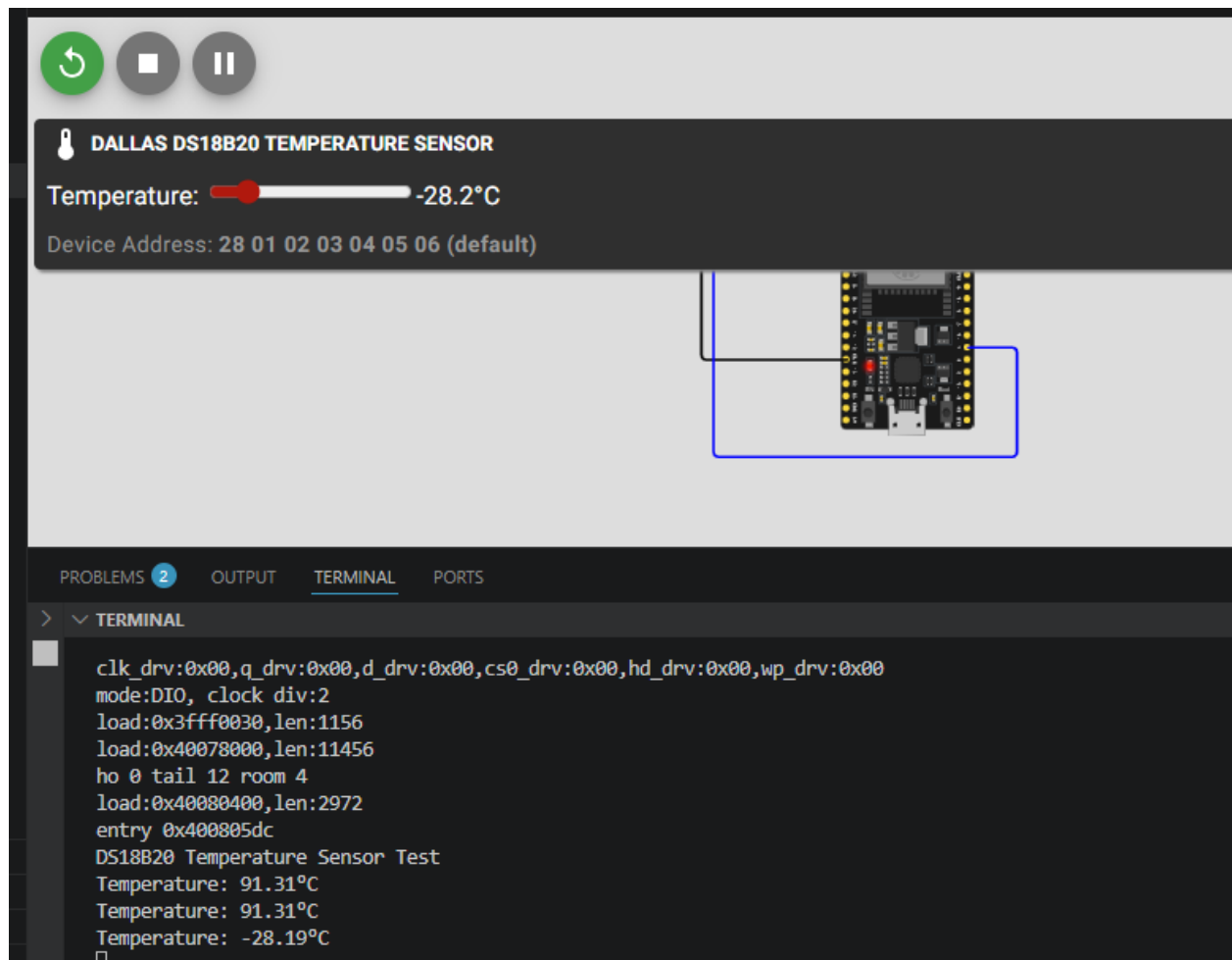
void loop() {
  sensors.requestTemperatures(); // Send the command to get temperatures

  // We use the function ByIndex, and 0 refers to the first sensor on the wire
  float tempC = sensors.getTempCByIndex(0);

  // Check if reading was successful
  if(tempC != DEVICE_DISCONNECTED_C) {
    Serial.print("Temperature: ");
    Serial.print(tempC);
    Serial.println("°C");
  } else {
    Serial.println("Error: Could not read temperature data");
  }

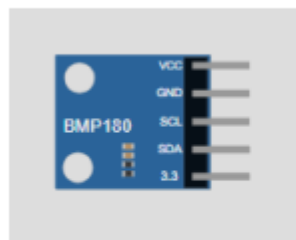
  delay(2000);
}

```

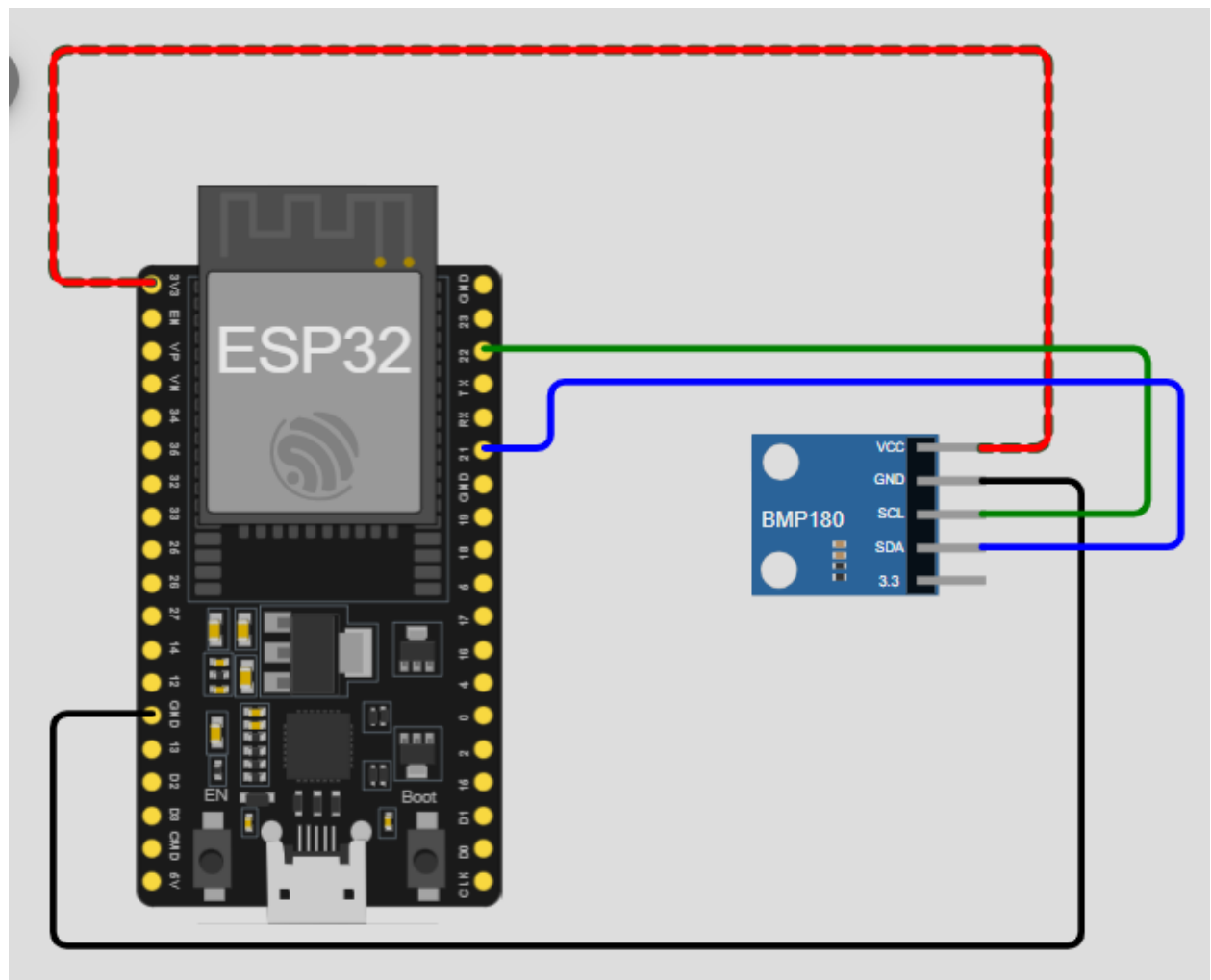


04.BMP180 (Barometric Pressure & Temperature Sensor)

Measures atmospheric pressure and temperature. It can also calculate altitude. Commonly used in weather forecasting, altitude tracking (drones, hiking devices), and environmental monitoring.



Pins on Sensor	Pins on ESP32
VCC	3.3V
GND	GND
SCL	GPIO 22
SDA	GPIO 21
3.3V	-



```
#include <Arduino.h>
#include <Adafruit_BMP085.h>

Adafruit_BMP085 bmp;

void setup() {
  Serial.begin(115200);

  if (!bmp.begin()) {
    Serial.println("Could not find a valid BMP180 sensor, check wiring!");
    while (1) {}
  }
}

void loop() {
  Serial.print("Temperature = ");
```

```

Serial.print(bmp.readTemperature());
Serial.println(" *C");

Serial.print("Pressure = ");
Serial.print(bmp.readPressure());
Serial.println(" Pa");

// Calculate altitude assuming 'standard' barometric pressure
// of 1013.25 millibar = 101325 Pascal
Serial.print("Altitude = ");
Serial.print(bmp.readAltitude());
Serial.println(" meters");

Serial.print("Pressure at sea level (calculated) = ");
Serial.print(bmp.readSeaLevelPressure());
Serial.println(" Pa");

Serial.println();
delay(2000);
}

```



BMP180 Barometric Pressure Sensor



PROBLEMS OUTPUT TERMINAL PORTS

> ▾ TERMINAL

```

Temperature = -5.60 *C
Pressure = 36999 Pa
Altitude = 7733.76 meters
Pressure at sea level (calculated) = 36999 Pa

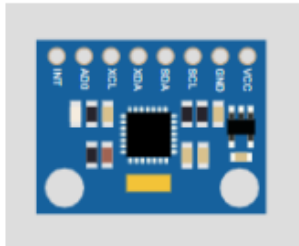
Temperature = 48.70 *C
Pressure = 36999 Pa
Altitude = 7733.76 meters
Pressure at sea level (calculated) = 36999 Pa

```

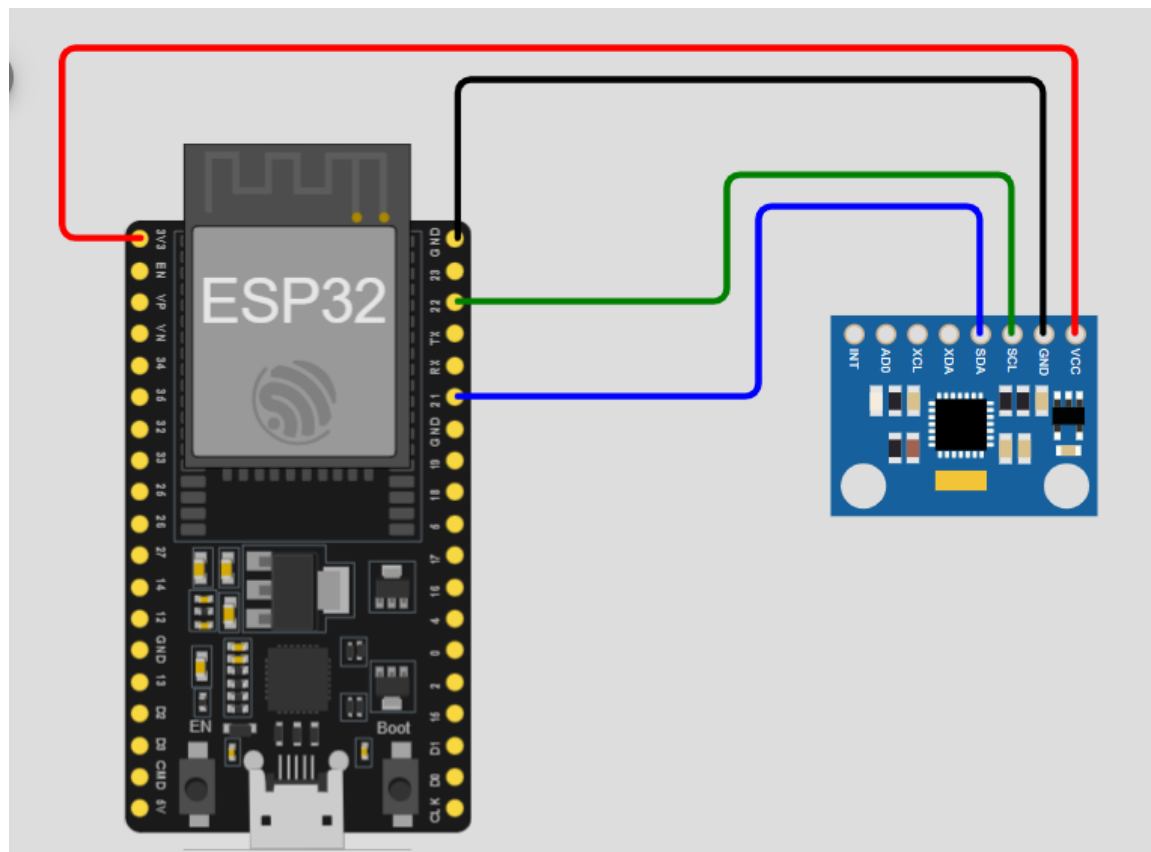


05.MPU6050 (3-Axis Accelerometer + Gyroscope + Temperature)

Detects motion, orientation, acceleration, and rotation. Essential for robotics, drones, self-balancing devices, gesture control, gaming controllers, and motion tracking applications.



Pins on Sensor	Pins on ESP32
INT	-
AD0	-
XCL	-
XDA	-
SDA	GPIO 21
SCL	GPIO 22
GND	GND
VCC	3.3V



```

#include <Arduino.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <Wire.h>

Adafruit_MPU6050 mpu;

void setup(void)
{
    Serial.begin(115200);

    // Try to initialize!
    if (!mpu.begin())
    {
        Serial.println("Failed to find MPU6050 chip");
        while (1)
        {
            delay(10);
        }
    }
    Serial.println("MPU6050 Found!");

    // Set sensor ranges
    mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
    mpu.setGyroRange(MPU6050_RANGE_500_DEG);
    mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);

    delay(100);
}

void loop()
{
    /* Get new sensor events with the readings */
    sensors_event_t a, g, temp;
    mpu.getEvent(&a, &g, &temp);

    /* Print out the values */
    Serial.print("Accel X: ");
    Serial.print(a.acceleration.x);
    Serial.print(", Y: ");
    Serial.print(a.acceleration.y);
    Serial.print(", Z: ");
    Serial.print(a.acceleration.z);
    Serial.println(" m/s^2");
}

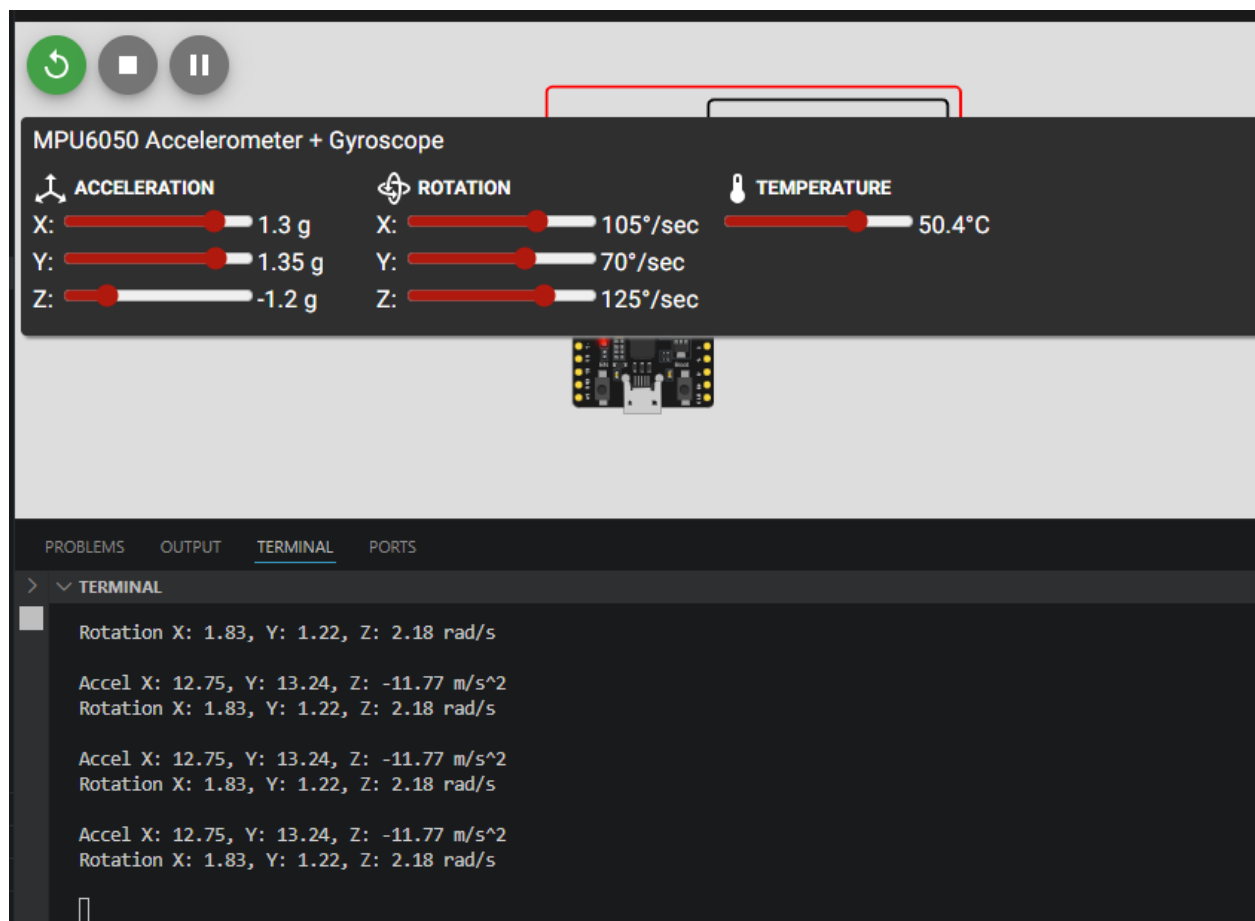
```

```

Serial.print("Rotation X: ");
Serial.print(g.gyro.x);
Serial.print(", Y: ");
Serial.print(g.gyro.y);
Serial.print(", Z: ");
Serial.print(g.gyro.z);
Serial.println(" rad/s");

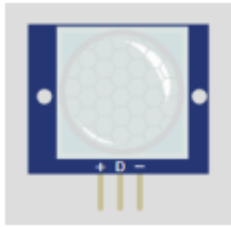
Serial.println("");
delay(500);
}

```

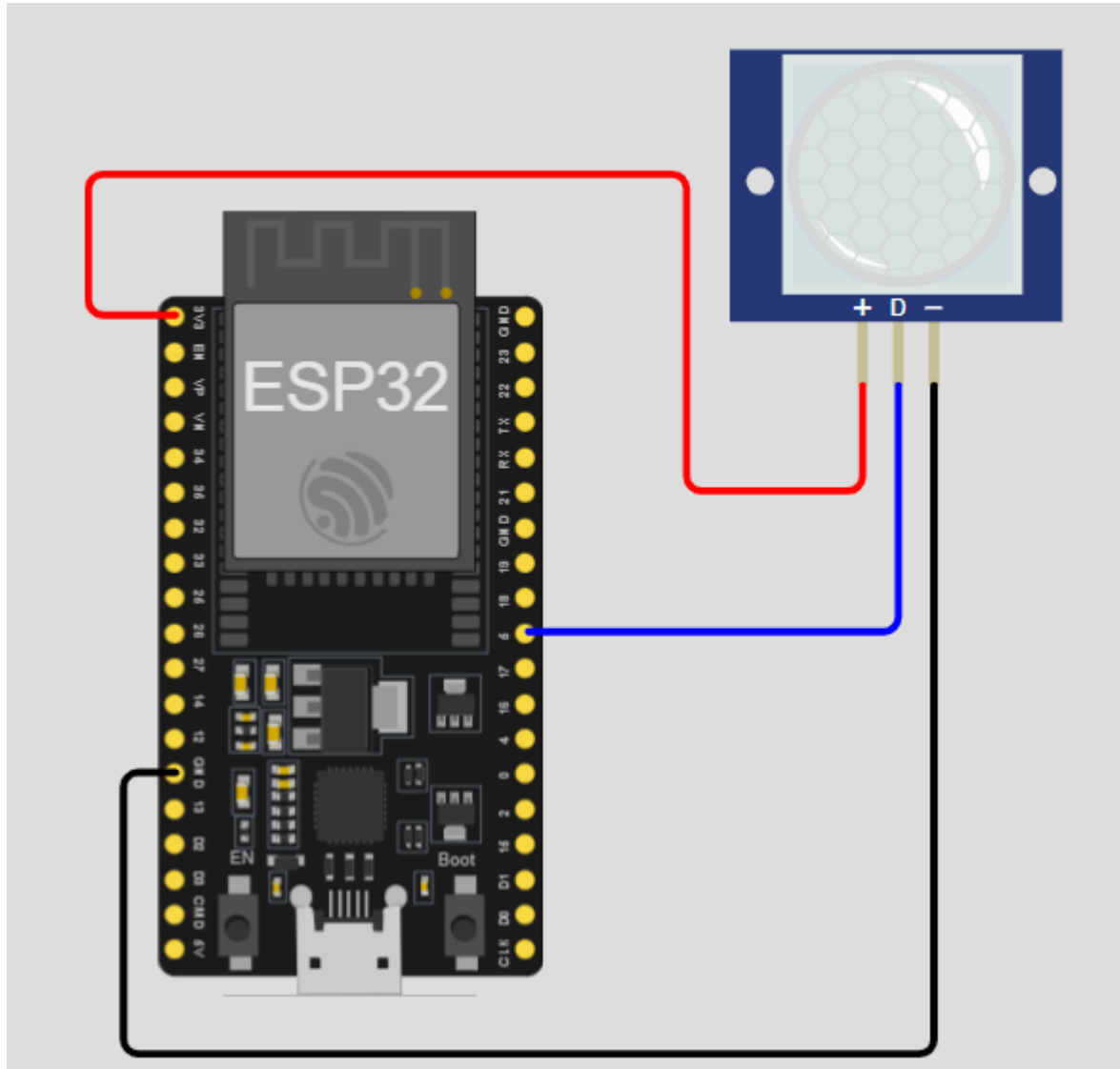


06.PIR Motion Sensor (Passive Infrared)

Detects movement of people or animals by sensing changes in infrared (heat) radiation. Primarily used in security alarms, automatic lighting, and presence detection systems. Digital output is HIGH on motion.



Pins on Sensor	Pins on ESP32
VCC	3.3V
OUT	GPIO 5
GND	GND



```
#include <Arduino.h>
#define PIR_PIN 5

void setup() {
  Serial.begin(115200);
```

```

pinMode(PIR_PIN, INPUT);
Serial.println("PIR Sensor Warm-up (takes ~30-60 seconds)...");

// PIR sensors need a warm-up period to calibrate to the room
delay(30000);
Serial.println("PIR Sensor Active!");
}

void loop() {
  int motionState = digitalRead(PIR_PIN);

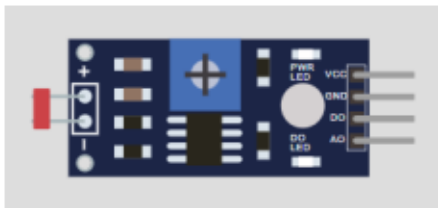
  if (motionState == HIGH) {
    Serial.println("--- MOTION DETECTED! ---");
  } else {
    Serial.println("No motion.");
  }

  delay(500); // Small delay to avoid flooding the Serial Monitor
}

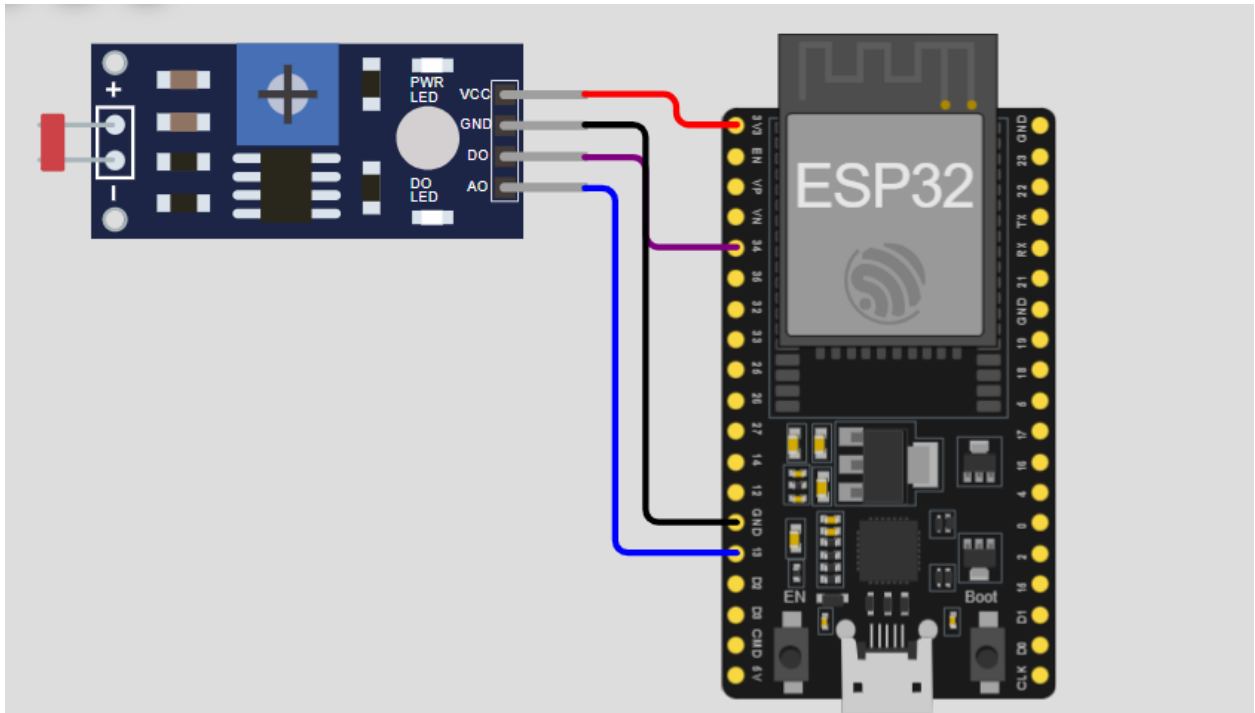
```

07.Photoresistor (LDR) Light Sensor

Detects light intensity by changing its resistance based on light levels. Used in automatic street lights, night lamps, camera exposure control, and light-activated systems.



Pins on Sensor	Pins on ESP32
VCC	3.3V
GND	GND
DO	-
AO	GPIO 34



```
#include <Arduino.h>

#define LDR_ANALOG_PIN 34
#define LDR_DIGITAL_PIN 13

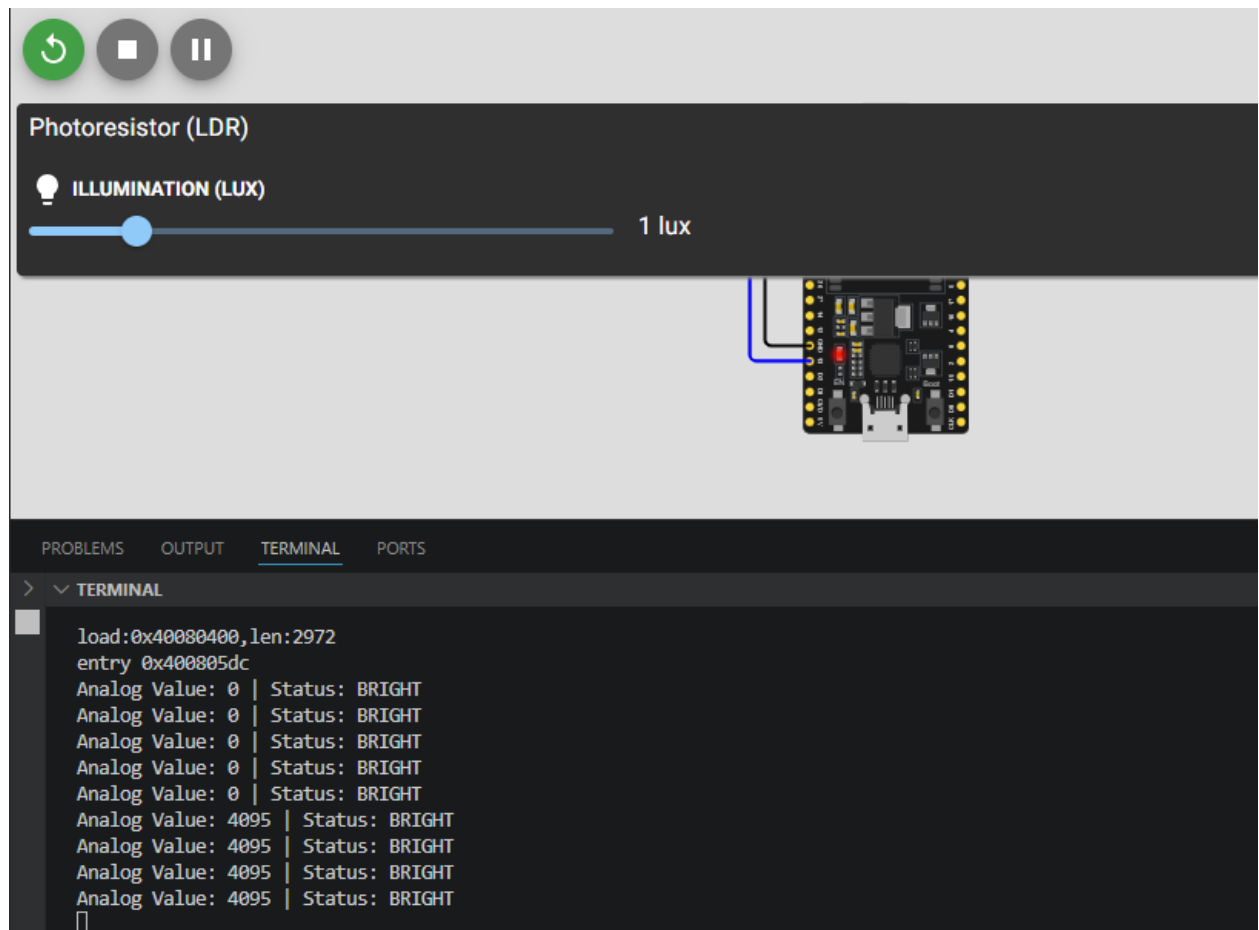
void setup() {
  Serial.begin(115200);
  pinMode(LDR_DIGITAL_PIN, INPUT);
}

void loop() {
  int analogVal = analogRead(LDR_ANALOG_PIN);
  int digitalVal = digitalRead(LDR_DIGITAL_PIN);

  Serial.print("Analog Value: ");
  Serial.print(analogVal); // 0 (Bright) to 4095 (Dark) usually

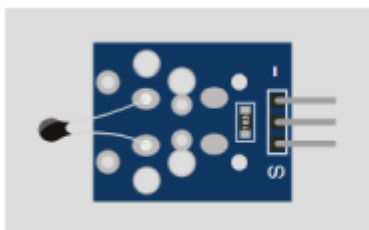
  if (digitalVal == HIGH) {
    Serial.println(" | Status: DARK (Threshold Reached)");
  } else {
    Serial.println(" | Status: BRIGHT");
  }

  delay(500);
}
```

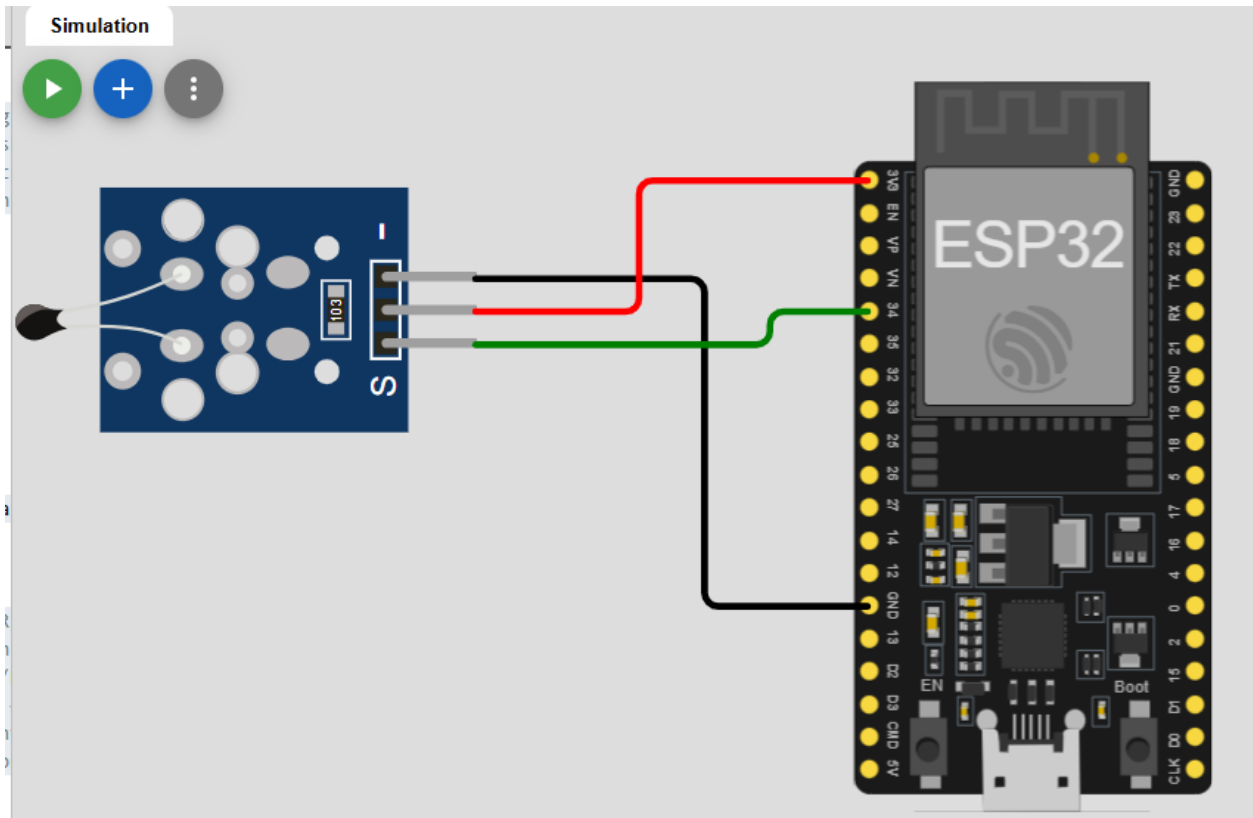


08.Analog Temperature Sensor (NTC Thermistor)

Measures temperature by changing resistance with heat (Negative Temperature Coefficient). Used for temperature monitoring and control in appliances, battery management, and general environmental sensing.



Pins on Sensor	Pins on ESP32
GND	GND
VCC	3.3V
OUT	GPIO 34



```
#include <Arduino.h>

#define THERMISTOR_PIN 34
#define SERIES_RESISTOR 10000 // 10k resistor
#define NOMINAL_RESISTANCE 10000 // resistance at 25 degrees C
#define NOMINAL_TEMPERATURE 25 // temp for nominal resistance
#define B_COEFFICIENT 3950 // The beta coefficient (usually 3435 or 3950)
#define ADC_MAX 4095 // ESP32 ADC resolution

void setup() {
  Serial.begin(115200);
}

void loop() {
  int adcValue = analogRead(THERMISTOR_PIN);

  // Calculate resistance
  float resistance;
  resistance = SERIES_RESISTOR / (ADC_MAX / (float)adcValue - 1);

  // Steinhart-Hart Equation
  float steinhart;
  steinhart = resistance / NOMINAL_RESISTANCE; // (R/Ro)
```

```

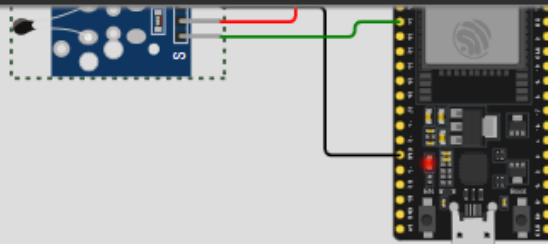
steinhart = log(steinhart);           // Ln(R/Ro)
steinhart /= B_COEFFICIENT;           // 1/B * Ln(R/Ro)
steinhart += 1.0 / (NOMINAL_TEMPERATURE + 273.15); // + (1/To)
steinhart = 1.0 / steinhart;          // Invert
steinhart -= 273.15;                  // convert to C

Serial.print("Temperature: ");
Serial.print(steinhart);
Serial.println(" °C");

delay(1000);
}

```

NTC Temperature Sensor (analog)
 Temperature:  44.9°C



PROBLEMS OUTPUT TERMINAL PORTS

> ▾ TERMINAL

```

configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
Temperature: 24.00 °C
Temperature: 44.91 °C
Temperature: 44.91 °C

```

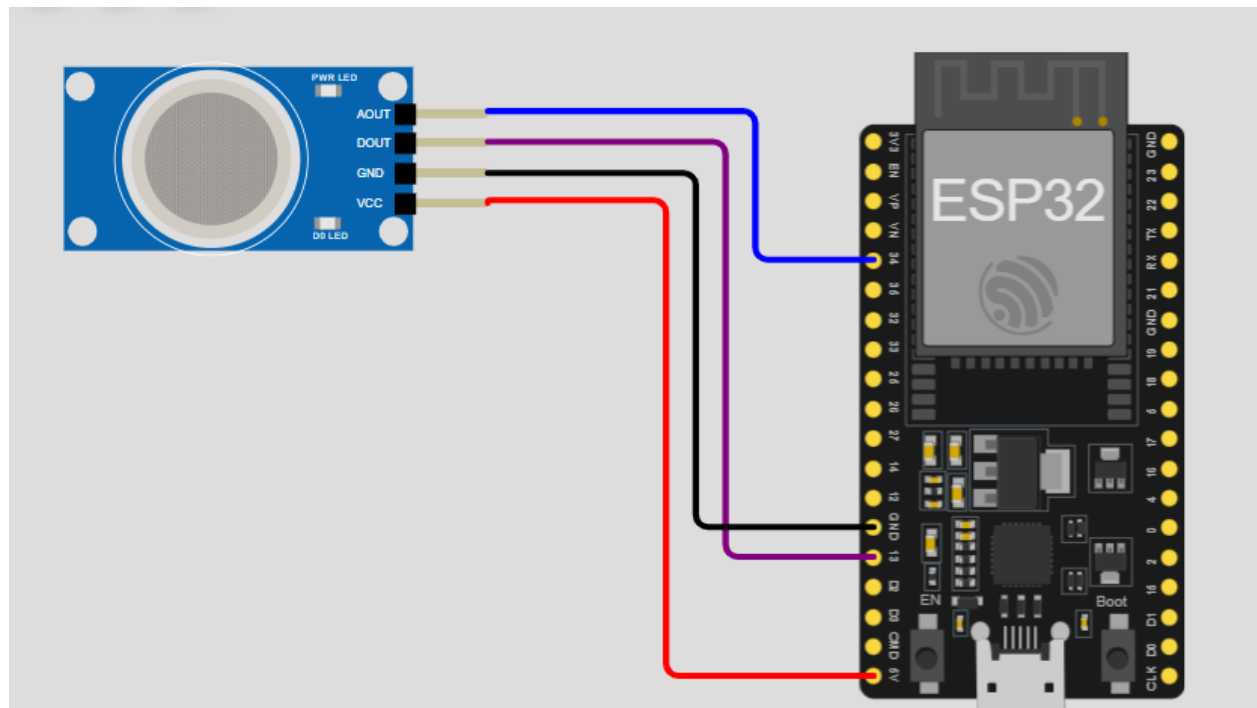
Default (08-L02-Task08-NTC-Thermistor) Auto Issue Ready

09.MQ2 Gas Sensor

Detects combustible gases such as LPG, methane, propane, and smoke. Commonly used in gas leakage alarms, air quality monitors, and safety systems.



Pins on Sensor	Pins on ESP32
AOUT	GPIO 34
DOUT	-
GND	GND
VCC	5V



```
#include <Arduino.h>

#define MQ2_ANALOG_PIN 34
#define MQ2_DIGITAL_PIN 13

void setup() {
  Serial.begin(115200);
  pinMode(MQ2_DIGITAL_PIN, INPUT);

  Serial.println("MQ2 Sensor Warming Up...");
  // It takes a moment for the internal heater to stabilize
```

```

    delay(20000);
}

void loop() {
    int analogValue = analogRead(MQ2_ANALOG_PIN);
    int digitalValue = digitalRead(MQ2_DIGITAL_PIN);

    Serial.print("Gas Level (Analog): ");
    Serial.print(analogValue);

    // Lower values usually mean clean air; higher values mean gas detected
    if (digitalValue == LOW) { // MQ series often use Active-Low for Digital Out
        Serial.println(" | [ ALERT: Gas/Smoke Detected! ]");
    } else {
        Serial.println(" | [ Air Status: Normal ]");
    }

    delay(1000);
}

```

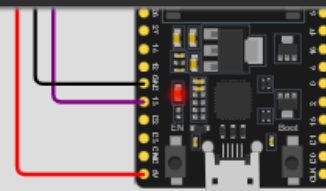


Gas Sensor



GAS (PPM)

1 ppm



PROBLEMS OUTPUT TERMINAL PORTS

> ▾ TERMINAL

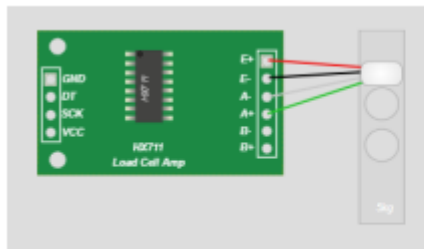
```

load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
MQ2 Sensor Warming Up...
Gas Level (Analog): 3932 | [ ALERT: Gas/Smoke Detected! ]
Gas Level (Analog): 3932 | [ ALERT: Gas/Smoke Detected! ]
Gas Level (Analog): 3932 | [ ALERT: Gas/Smoke Detected! ]
Gas Level (Analog): 1675 | [ Air Status: Normal ]
Gas Level (Analog): 1675 | [ Air Status: Normal ]
Gas Level (Analog): 1675 | [ Air Status: Normal ]

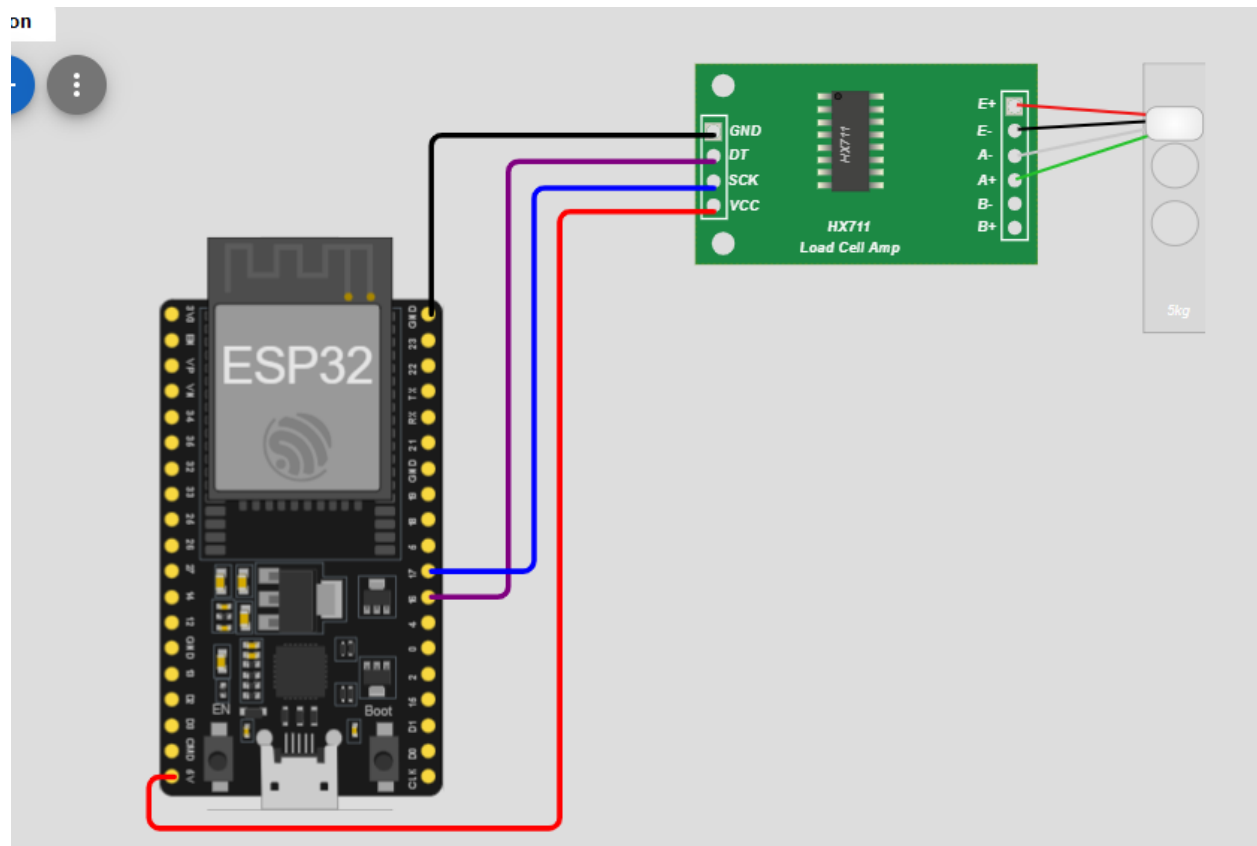
```

10.HX711 Load Cell Amplifier

Amplifies tiny signals from load cells (strain gauges) to accurately measure weight. Used in digital scales, industrial weighing systems, and force/weight monitoring projects.



Pins on Sensor	Pins on ESP32
GND	GND
DT	GPIO 16
SCK	GPIO 17
VCC	5V



```
#include <Arduino.h>
#include "HX711.h"

// HX711 circuit wiring
```

```

const int LOADCELL_DOUT_PIN = 16;
const int LOADCELL_SCK_PIN = 17;

HX711 scale;

void setup()
{
  Serial.begin(115200);
  Serial.println("HX711 Demo");

  scale.begin(LOADCELL_DOUT_PIN, LOADCELL_SCK_PIN);

  Serial.println("Before setting up the scale:");
  Serial.print("read: \t\t");
  Serial.println(scale.read()); // print a raw reading from the ADC

  scale.set_scale(); // Reset the scale to 1
  scale.tare();      // Reset the scale to 0

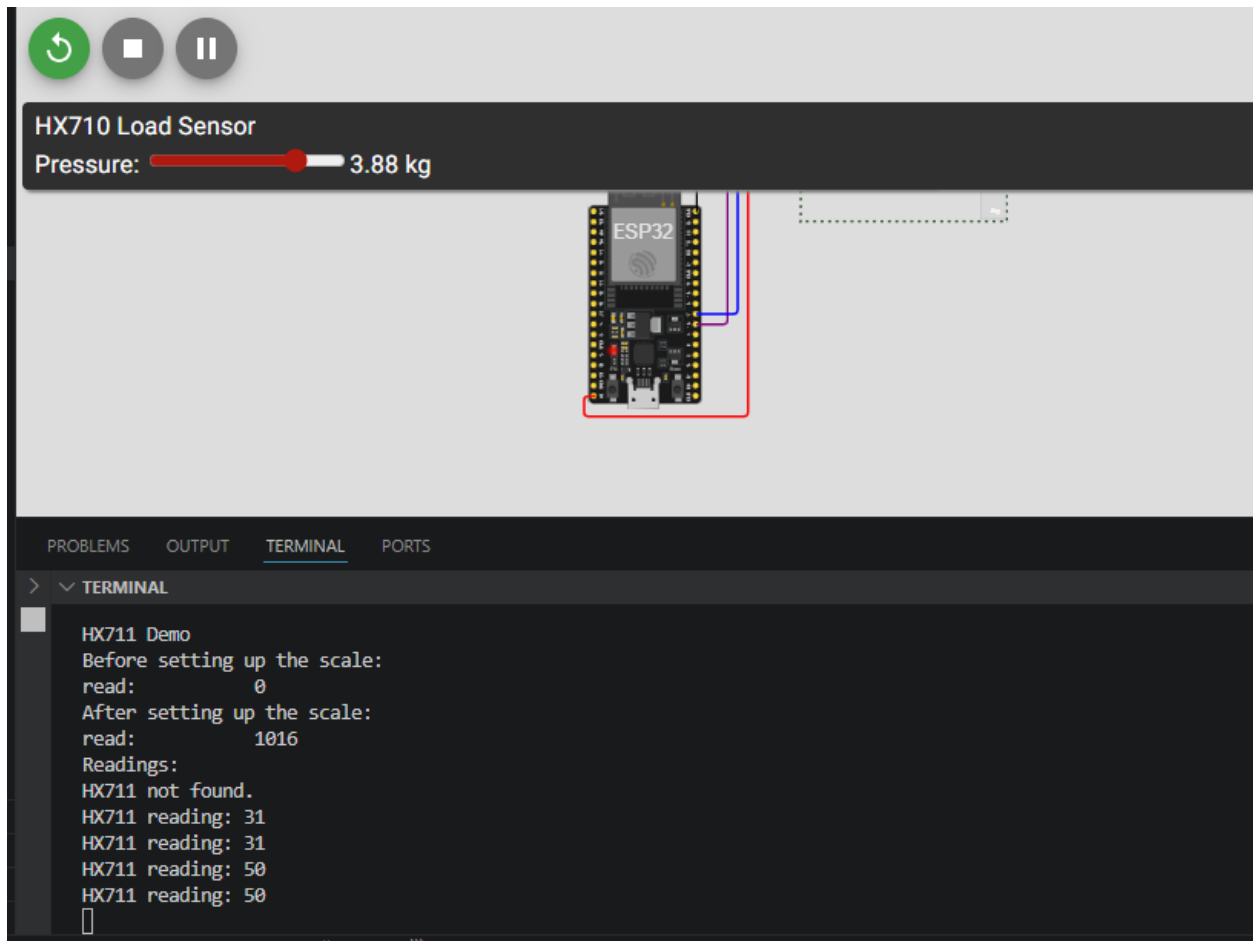
  Serial.println("After setting up the scale:");
  Serial.print("read: \t\t");
  Serial.println(scale.read()); // print a raw reading from the ADC

  Serial.println("Readings:");
}

void loop()
{
  if (scale.is_ready())
  {
    long reading = scale.read();
    Serial.print("HX711 reading: ");
    Serial.println(reading);
  }
  else
  {
    Serial.println("HX711 not found.");
  }

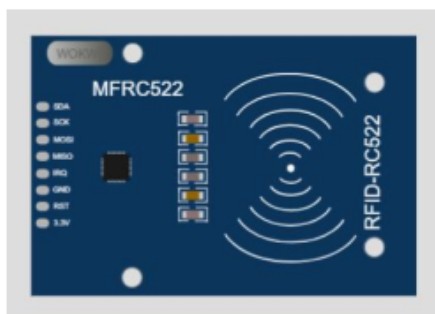
  delay(1000);
}

```

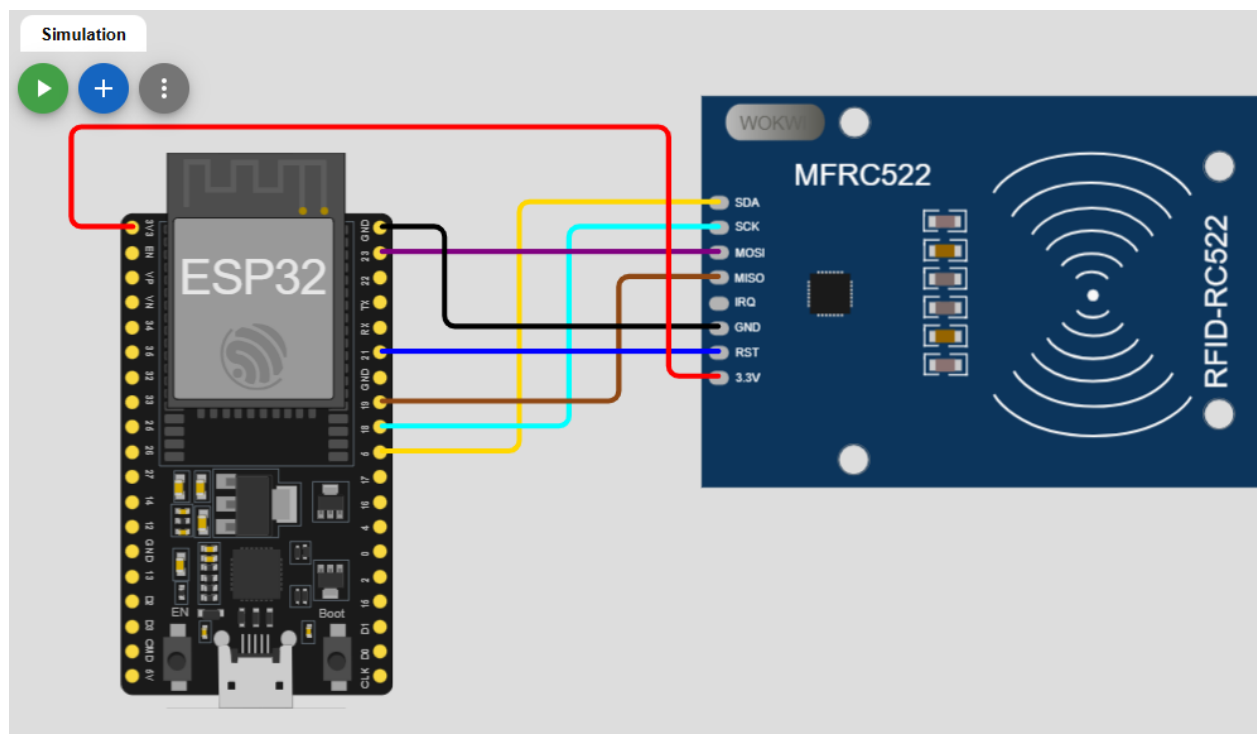


11.MFRC522 RFID/NFC Reader

Reads and writes RFID/NFC cards and tags for identification. Used in access control systems, attendance tracking, contactless payments, and inventory management.



Pins on Sensor	Pins on ESP32
SDA	GPIO 5
SCK	GPIO 18
MOSI	GPIO 23
MISO	GPIO 19
IRQ	-
GND	GND
RST	GPIO 21
3.3V	3.3V



```
#include <Arduino.h>
#include <SPI.h>
#include <MFRC522.h>

#define SS_PIN    5
#define RST_PIN   21

MFRC522 mfrc522(SS_PIN, RST_PIN); // Create MFRC522 instance

void setup() {
  Serial.begin(115200); // Initialize serial communications
  SPI.begin();          // Init SPI bus
  mfrc522.PCD_Init();   // Init MFRC522

  Serial.println("Scan an RFID tag or card...");
}

void loop() {
  // Look for new cards
  if ( ! mfrc522.PICC_IsNewCardPresent()) {
    return;
  }
}
```

```

// Select one of the cards
if ( ! mfrc522.PICC_ReadCardSerial()) {
    return;
}

// Show UID on serial monitor
Serial.print("Tag UID:");
String content= "";
for (byte i = 0; i < mfrc522.uid.size; i++) {
    Serial.print(mfrc522.uid.uidByte[i] < 0x10 ? " 0" : " ");
    Serial.print(mfrc522.uid.uidByte[i], HEX);
    content.concat(String(mfrc522.uid.uidByte[i] < 0x10 ? " 0" : " "));
    content.concat(String(mfrc522.uid.uidByte[i], HEX));
}

Serial.println();
Serial.print("Message : ");
content.toUpperCase();

// Example: Check for a specific authorized UID
if (content.substring(1) == "XX XX XX XX") { // Change this to your tag's UID
    Serial.println("Access Authorized");
} else {
    Serial.println("Access Denied");
}

delay(1000);
}

```



MFRC522 RFID Reader

Card



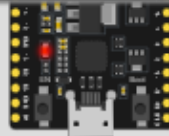
Yellow Card ▾

TAP



Hold

UID: 55:66:77:88



PROBLEMS

OUTPUT

TERMINAL

PORTS

> ▾ TERMINAL

Scan an RFID tag or card...

Tag UID: 11 22 33 44

Message : Access Denied

Tag UID: C0 FF EE 99

Message : Access Denied

Tag UID: C0 FF EE 99

Message : Access Denied

Tag UID: 04 11 22 33

Message : Access Denied

Tag UID: 55 66 77 88

Message : Access Denied

█

